AI Engineer 자기소개서

황화인

목차

1. Copilot ..... p. 2
실제 실행 결과로 성공을 판정했습니다

2. Q&A ..... p. 3
파일 처리 오류를 해결하고 Agent의 작업 기준을 관리했습니다

3. ALL_FOR_ONE ..... p. 4
현업 요구에 맞춰 보고서 생성과 분석 구성을 개선했습니다

4. MEMORIA LABYRINTH ..... p. 5
발화 주체를 보존하는 기억 시스템을 직접 구현했습니다

5. 회사 비교실험과 SENTINEL ..... p. 6
품질 목표의 효과와 시간 부담을 비교했습니다

목차 항목을 클릭하면 해당 위치로 이동합니다.

세부 항목은 PDF 책갈피에서, 목차 복귀는 각 페이지 하단에서 이용할 수 있습니다.

AI Engineer

자기소개서

황 화 인

포트폴리오(PPT) | 포트폴리오(Notion) | 이력서 | 개발블로그 | 링크드인

저는 AI가 만든 결과를 실제 실행과 데이터로 확인하는 AI Engineer입니다. 베스핀글로벌에서는 자동화 빌더 에이전트인 Copilot의 실행 결과와 코딩 Agent의 수정 작업을 검증했고, 부동산 보고서와 게임 NPC 프로젝트에서는 생성이 멈추거나 기억을 잘못 꺼내는 원인을 찾아 처리 구조를 바꿨습니다.

1. Copilot:

실제 실행 결과로 성공을 판정했습니다

베스핀글로벌에서 Helpnow Foundry Copilot의 개선을 맡았을 때, 직접 사용해보니 화면에는 워크플로우가 만들어져도 실제로 작동하지 않거나 수정 요청이 반영되지 않는 경우가 있었습니다. Copilot은 "결재완료 메일이 오면 내용을 시트에 적재해줘" 같은 요청을 받아 Gmail 읽기·시트 쓰기 블록을 연결하는 Agent입니다. 당시에는 발견한 버그를 개별적으로 확인해야 했고, 수정 전보다 나아졌는지 비교할 기준도 없었습니다. 저는 같은 요청을 다시 실행하고 실제 결과까지 확인하는 평가 하네스를 만들었습니다.

고객 컴플레인 정리 문서조차 없던 환경에서 오류 재발을 막고자 실제 고객 빈도가 높은 요청 패턴 100개 시나리오를 설정하고, 외부 API 응답과 DB 실제 저장 상태를 End-to-End로 검증하는 277개 사후조건(Post-condition)을 설계해 정량적 기준선(Baseline)을 확립했습니다. 예를 들어 포상 대상자에게 메일을 보내는 시나리오에서는 대상자 발송 여부, 비대상자 미발송, 시트 발송 내역 적재를 각각 검사했습니다. Gmail API는 실행 시각과 수신자로 메일을 다시 조회하고, 시트는 시험 자료의 고유번호로 결과를 읽어 과거 메일이나 타 시험 기록을 이번 성공으로 집계하는 오판을 방지했습니다.

평가 결과를 추적하다 보니 배포를 시도하지 않은 22건도 통과로 집계되고 있었습니다. 평가기가 실패 응답이 없다는 이유로 성공 처리한 것이 원인이었습니다. 저는 미시도·성공·서버 응답 실패·실행 예외를 분리하고, 연결 검사도 제품이 보정하기 전 계획 대신 DB에 실제 저장된 연결을 읽도록 수정했습니다. 기능 합격은 필수 조건을 코드로 판정하는 결정론적 검사와 외부 결과 확인으로 결정했습니다. LLM Judge가 매기는 설명 품질 점수는 참고값으로 분리해, 설명이 그럴듯해도 실행에 실패한 작업이 합격하지 않도록 했습니다.

다음 과제는 이 기준선 위에서 AI 코딩 에이전트가 결함을 자율적으로 추적하고 수정하도록 만드는 하네스 파이프라인의 구축이었습니다. 세션마다 맥락을 잊고 구식 스크립트를 임의 실행하거나 서버 다운을 제품 결함으로 오인하는 문제를 막고자 세션 진입 훅(SessionStart)을 구축했습니다. 에이전트가 `AGENTS.md → spec.md → repair-loop.md` 순서로 읽도록 동선을 짜고, `repair-loop.md`를 표준 오답 노트로 두어 제품 로직 결함과 환경 오류를 먼저 엄격히 분류하도록 통제했습니다. 아울러 세션 진입 즉시 서버 가동 상태와 직전 런 실패 지표, 미커밋 수정 문서(`fixes/`) 잔여 건수를 브리핑하여, 이렇게 서버 다운을 제품 버그로 오진하는 헛수고를 사전에 차단했습니다. 나아가 `PreToolUse` 훅으로 시험지 변조와 구식 평가기 호출을 차단하고 서버 관리 스크립트 사용을 강제했습니다.

2. Q&A:

파일 처리 오류를 해결하고 Agent의 작업 기준을 관리했습니다

같은 회사에서 맡은 Q&A 기능은 Google Chat에 흩어진 고객 문의를 서비스 안에서 등록하고 기존 답변을 찾아보게 하는 작업이었습니다. 저는 멘토와 요구사항·정책을 협의하고, 기존 로그인과 공통 UI에 화면 6개와 FastAPI API 12개, 문의·답변·첨부 테이블을 연결했습니다. 첨부파일을 시험하다가 정상 파일 다음에 5MB 초과 파일을 넣으면 요청은 실패하고 DB 변경도 롤백되지만, 첫 파일은 파일 저장소인 MinIO에 남는 것을 확인했습니다. 실제로 저장소 파일 수가 35개에서 36개로 늘었습니다.

기존 코드는 파일마다 검증과 업로드를 반복하고 있었습니다. 저는 모든 파일의 개수·확장자·용량을 먼저 검사한 뒤 업로드를 시작하도록 순서를 바꿨습니다. 선검증을 통과해도 업로드나 DB 커밋은 실패할 수 있으므로, 해당 요청에서 올린 파일을 별도로 삭제하는 보상 처리도 넣었습니다. 삭제에 실패한 파일의 키는 로그에 남겼습니다. 그리고 같은 용량 초과 시나리오를 다시 실행했을 때 저장소 파일 증가량이 1개에서 0개로 줄어드는 것을 확인했습니다.

요구사항이 바뀔 때 코딩 Agent가 과거 설계를 구현하는 문제도 예방하려 했습니다. 원본 엑셀과 설계서를 기능별 작업 문서로 추출하되, 새 세션에서 원본 수정 시각과 추출본 생성 시각을 비교해 갱신이 필요하면 안내했습니다. Jira 티켓에는 수정할 코드 위치와 허용 범위, 재현 절차, 완료 조건을 적었습니다. 요구사항은 티켓 본문에, 구현·검증 결과는 코멘트에 남겨 문서를 다시 동기화해도 작업 이력이 덮어써지지 않게 했습니다.

커밋·푸시·PR 규칙은 해당 명령을 실행하기 직전에 다시 전달했습니다. 세션 초반의 안내가 긴 대화 속에서 누락되는 것을 막기 위해, PreToolUse 훅이 명령을 감지하면 규칙 원본을 읽고 필수 항목과 사전 점검 목록을 Agent에 전달하도록 했습니다. 규칙을 훅 코드에 복사하면 문서와 코드를 따로 고쳐야 하므로, 실행 시점에 원본을 읽는 방식을 택했습니다.

3. ALL_FOR_ONE:

현업 요구에 맞춰 보고서 생성과 분석 구성을 개선했습니다

외부기관 협업 프로젝트 ALL_FOR_ONE에서는 부동산 현업자가 신뢰할 분양성 검토 보고서를 만들고자 했습니다. 저는 부동산 마케팅 기획 경험을 바탕으로 한국부동산마케팅협회에 자문을 구하고 실무 보고서 7건을 분석했습니다. 보고서 작성에는 1\~2주가 걸렸고, 기존 자동화 결과는 데이터를 나열해도 해당 사업지와 시점에 맞는 해석이 부족하다는 의견을 들었습니다. 저는 출처와 판단 근거를 함께 제공하도록 DB와 처리 구조를 설계하고, LangGraph로 역할별 Agent를 연결했습니다. 검색은 pgvector의 의미 검색과 PGroonga의 키워드 검색을 결합해 정책의 정확한 수치와 표현이 다른 유사 정책을 함께 찾도록 구성했습니다.

마감이 앞당겨졌을 때는 의뢰자에게 가장 필요한 결과를 다시 물었습니다. 정책 보고서가 우선이라는 답을 듣고, 정책 분석과 각 Agent의 결과를 엮는 최종 보고서에 개발 시간을 집중했습니다. 보고서를 네 구간으로 나눠 작성하던 중에는 네 번째 호출에서 입력 한도를 넘겨 생성이 중단됐습니다. 메시지 누적 과정을 확인하니 약 7만 토큰의 기초자료가 매 호출에 다시 들어가 28만 토큰까지 쌓이고 있었습니다. 저는 첫 호출에만 전체 자료를 넣고, 이후에는 기존 메시지를 유지하면서 이전 구간 요약과 작성 지시만 추가하도록 바꿨습니다. 입력은 약 8만 토큰으로 줄었고 보고서를 끝까지 생성했습니다.

논문에서 효과가 보고된 ReAct가 부동산 분석에도 도움이 되는지는 제 환경에서 확인해야 한다고 판단했습니다. 저는 추론과 도구 실행을 반복하는 ReAct 구조에 자기검증을 결합하고, 전체 적용·관련 구성 제거·역할별 선택 적용의 세 구성을 DeepEval 9개 지표로 비교했습니다. 관련 구성을 제거했을 때 단순 추출형 Agent 두 개는 평가 점수가 높아졌지만, 복합 분석형에서는 낮아지는 경우가 나타났습니다. 저는 이 결과를 바탕으로 Agent의 역할에 따라 적용 범위를 나눴습니다.

선택 적용 후에도 주변시장 Agent의 점수가 떨어져 출력을 확인했습니다. 모델이 자기검증 도구를 실제로 호출하지 않고, 호출할 내용을 평문으로 출력하면서 내부 점검 문장이 보고서에 섞이고 있었습니다. 저는 ReAct 루프 뒤에 출력 정리 단계를 추가해 해당 문장을 제거하고, Structured Output의 result 항목에서 보고서 본문만 추출했습니다. 역할별 선택 적용과 출력 경로 수정을 반영한 구성의 종합 평가 점수는 79.04%에서 83.52%로 높아졌습니다.

프로젝트는 현업 전문가의 검토를 거쳐 2025년 부동산산업의 날에서 시연했으며, 전시 내용은 중앙일보와 한국경제의 행사 보도에 소개됐습니다.

4. MEMORIA LABYRINTH:

발화 주체를 보존하는 기억 시스템을 직접 구현했습니다

MEMORIA LABYRINTH는 언리얼 개발자들과 협업해 만든 게임 프로젝트입니다. 저는 플레이어와 나눈 대화를 기억하고 관계 변화에 따라 다르게 반응하는 NPC의 대화·기억 기능을 맡았습니다. 초기에는 대화의 핵심 사실을 추출하고 중복 기억을 정리하는 Mem0를 적용했습니다. 그런데 NPC가 사용자의 정보를 자기 정보처럼 말해 DB를 확인하니, 당시 구성에는 "딸기를 좋아함"처럼 사실만 남고 누가 좋아하는지는 빠져 있었습니다. 플레이어의 말과 NPC의 말을 모두 기억해야 하는데 저장 단계에서 주체가 사라져, 검색한 기억을 응답에 넣어도 혼동이 이어졌습니다.

저는 Mem0의 기억 처리를 PostgreSQL 기반 장기기억 시스템으로 직접 구현해 대체했습니다. 대화에서 핵심 사실을 주어·관계·대상인 SPO 구조로 추출하고, 말한 사람과 그 사실의 주체를 함께 저장했습니다. 대화 원문은 세션 복원을 위해 별도 테이블에 보존하고, 기억 검색에는 추출한 사실만 사용하도록 나눴습니다. 취향이 바뀌면 이전 사실에 무효화 시점인 invalid_at을 기록해 원본은 남기되 현재 검색에서는 제외했습니다. "사과를 좋아해" 다음에 "사과를 싫어해"라고 말한 대화에서는 NPC가 최신 선호를 기준으로 답하는 것을 확인했습니다.

기억이 저장돼 있는데도 답하지 못하는 경우도 있었습니다. DB에는 포도를 좋아한다는 정보가 있었지만 "내가 좋아하는 과일이 뭐였지?"라고 물으면 회상하지 못했습니다. 저는 사실을 추출할 때 포도와 함께 과일·음식 같은 상위 범주 키워드를 저장하고, 이를 임베딩과 키워드 검색에 함께 반영했습니다. 매 질문마다 LLM을 추가 호출해 검색어를 확장하면 응답이 늦어질 수 있어 저장 단계에서 처리했습니다. 수정 후 사과 선호를 저장한 대화에서는 "좋아하는 과일"이라는 질문에 사과를 답하는 것을 확인했습니다.

언리얼 팀이 보낸 통신 규격에는 호감도만 있고 기억해금도가 없었습니다. 저는 이대로면 호감도가 떨어질 때 이미 되찾은 기억도 다시 잠기는 구조가 된다고 설명했습니다. 반대로 호감도가 떨어지지 않게 하면 플레이어의 말에 따라 NPC의 감정이 달라지는 경험을 보여주기 어려웠습니다. 대화에 따라 감정은 바뀌되 되찾은 기억은 유지한다는 기획에 맞춰, 팀과 두 값을 별도 파라미터로 나누기로 합의했습니다. 팀은 이 프로젝트로 최우수상인 1위를 받았습니다.

5. 회사 비교실험과 SENTINEL:

품질 목표의 효과와 시간 부담을 비교했습니다

회사에서 코딩 자동화 하네스를 비교할 때는 Agent가 완료했다고 말한 결과를 어떻게 검증할지부터 정했습니다. 백엔드 결함 3건을 수정 전 상태로 되돌리고, 모델·하네스 11개 구성을 총 99회 실행했습니다. 각 실행의 작업 환경을 분리하고, Agent가 정답 커밋과 숨긴 채점 테스트를 수정의 힌트로 참조하지 못하도록 접근을 제한했습니다. 저는 결함을 재현하는 테스트와 기존 테스트의 신규 실패·소실 여부로 기능 해결을 판정하고, Agent의 완료 선언과 대조했습니다. 코드 품질은 복잡도와 테스트 커버리지를 함께 보는 CRAP, 코드에 의도적인 변형을 넣어 테스트가 잡아내는지 확인하는 Mutation Testing으로 따로 측정했습니다. 이때 숨긴 채점 테스트는 품질 측정에서 제외하고, Agent가 수정한 코드와 작성한 테스트를 대상으로 삼았습니다.

이후에는 Prime Agent와 deepseek-v4-flash를 고정하고, 품질 목표와 수정·재측정 절차를 달리한 6개 조건을 54회 비교했습니다. 모든 조건에 기존 기능의 이상 여부를 확인하는 회귀 테스트 작성·실행을 지시했고, 품질 목표를 준 조건에는 검사 도구 사용법과 기준 미달 시 수정·재측정 절차를 추가했습니다. 그중 한 조건에는 CRAP 8 이하·mutation 90% 이상을 목표로 주고, 다른 조건에는 CRAP 목표만 6 이하로 강화했습니다.

결과를 비교할 때는 두 조건 모두 기능 해결과 CRAP 8 이하·mutation 90% 이상을 함께 달성했는지 확인했습니다. 두 조건 모두 9회 중 5회 달성했지만 실행당 평균 시간은 각각 79.3분과 102.6분이었습니다. 저는 같은 달성 횟수에 시간 부담이 적었던 CRAP 8 이하·mutation 90% 이상을 기본 기준으로 선택했습니다.

이 검사 절차를 다른 프로젝트에서도 쓰기 위해 오픈소스 프로젝트 SENTINEL을 만들었습니다. Python·TypeScript·Java의 기존 검사 도구를 연결하고, 검사한 파일·함수와 범위, 점수, 기준 미달 위치를 공통 JSON으로 반환하도록 구현했습니다. 검사 대상이 없거나 도구 실행이 실패한 경우도 통과와 구분했습니다. SENTINEL이 진단 결과를 반환하면 Claude Code·Codex가 해당 코드와 테스트를 수정하고, 처음 요청한 범위와 기준으로 다시 검사하도록 플러그인 지침을 연결했습니다. 이렇게 제가 직접 확인한 실패를 검사 조건과 수정 절차로 남겨, 다음 작업에서도 같은 근거로 결과를 판단하도록 만들었습니다.

저는 새로운 AI 기능을 맡았을 때도 먼저 실제 입력과 기대 결과를 확인하겠습니다. 결과가 어긋나면 데이터가 만들어지는 단계, 도구가 실행되는 경로, 성공을 판단하는 기준까지 내려가 원인을 찾고, 선택한 해결책을 팀이 다시 검증할 수 있는 기록으로 남기겠습니다.